# CSP-203: Software Tools & Technologies Lab

## Chapter – 6

## Web Programming: Introduction to CSS & JavaScript (Lecture-2)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

# Contents


- Introduction
- Classes in CSS
- Types of CSS
- CSS Box Model
- CSS Bootstrap
- Introduction to JavaScript
- Variables in JS
- Conditional Statements in JS
- Loops in JS
- Arrays in JS
- Function in JS
- JavaScript with HTML

# Introduction

- CSS stands for **Cascading Style Sheets**.
- It is used to **control the appearance and layout** of HTML documents.
- CSS defines **styles** such as colors, fonts, spacing, borders, and positioning.
- CSS can be written in three ways:
  - **Inline CSS** (inside HTML tags e.g., <p style=font-size:20px; color:red")
  - **Internal CSS** (inside <style> tag within HTML)
  - **External CSS** (linked via .css file)
- The term "cascading" means styles are applied based on **priority and inheritance rules**.

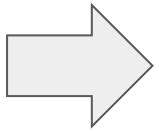


# Example - 1

```html
<!DOCTYPE html>
<html>
<head>
  <title>Welcome</title>
  <style>
    h1 {
      color: red; /* All h1 tags will appear in red */
    }
  </style>
</head>
<body>

  <h1>IIT BHILAI</h1>
  The sentence "IIT BHILAI" will appear in red color.

</body>
</html>
```



**IIT BHILAI**

The sentence "IIT BHILAI" will appear in red color.

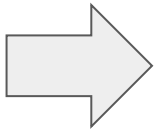
# Example - 2

```html
<!DOCTYPE html>
<html>
<head>
  <title>My Page</title>
  <style>
    h1, h2 {
      background-color: blue; /* Applies blue background to both h1 and h2 */
      color: white; /* Optional: makes the text readable */
    }
  </style>
</head>
<body>

  <h1>First Test</h1>
  <p>Here is the introduction</p>

  <h2>Contact page will be here</h2>
  <p>Email us at example@xyz.com</p>

</body>
</html>
```

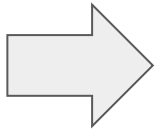


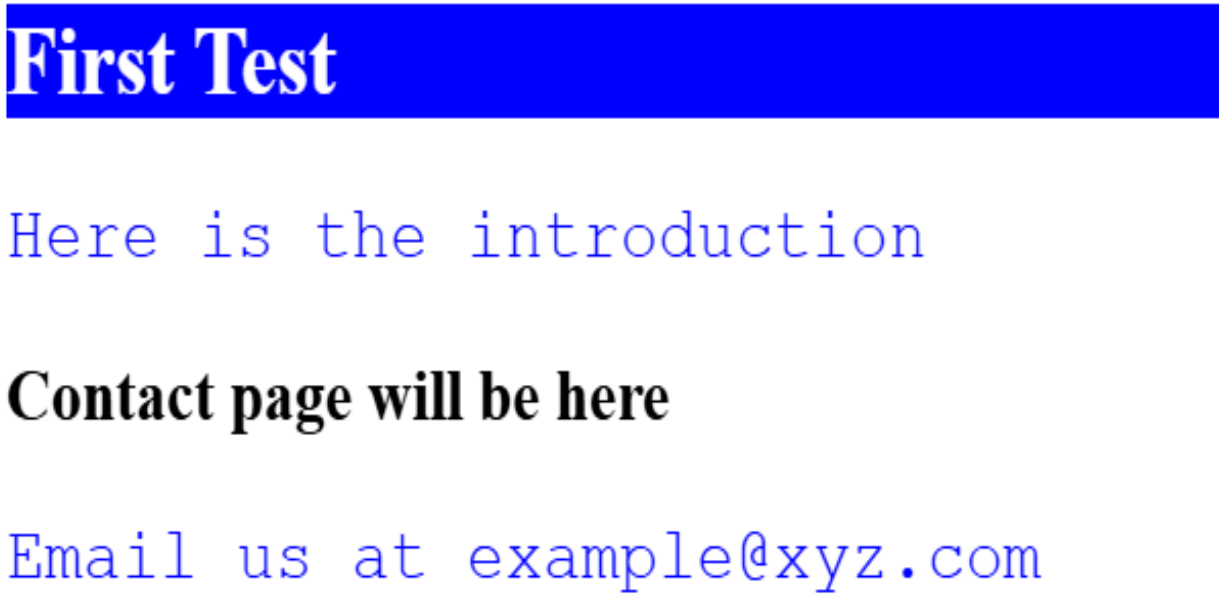


# Changing Font

```html
<!DOCTYPE html>
<html>
<head>
  <title>Changing Font</title>
  <style>
    h1 {
      background-color: blue;
      color: white; /* Optional: improves readability */
    }

    p {
      color: blue;
      font-size: 25px;
      font-family: Courier New;
    }
  </style>
</head>
<body>
  <h1>First Test</h1>
  <p>Here is the introduction</p>
  <h2>Contact page will be here</h2>
  <p>Email us at example@xyz.com</p>
</body>
</html>
```





# CSS Borders

- CSS border adds lines around elements.
- You can specify:
  - **Side**: border-top, border-bottom, border-left, border-right
  - **Width**: in pixels (1px, 2px, etc.)
  - **Style**: e.g., solid, dashed, dotted
  - **Color**: e.g., red, blue
- If no side is mentioned (border:), it applies to **all sides**.

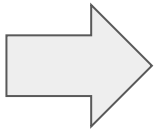
# Example

```html
<!DOCTYPE html>
<html>
<head>
  <title>CSS Borders Example</title>
  <style>
    h1 {
      border: 1px solid red; /* Border on all sides */
    }

    h2 {
      border-bottom: 2px solid blue; /* Border only at the bottom */
    }
  </style>
</head>
<body>

  <h1>This is a Heading 1 (Red Border)</h1>
  <h2>This is a Heading 2 (Blue Bottom Border)</h2>

</body>
</html>
```



This is a Heading 1 (Red Border)

This is a Heading 2 (Blue Bottom Border)

# Classes in CSS

- Classes let you apply different styles to elements of the **same tag**.
- Applied in HTML using class="classname".
- Defined using a dot . in CSS (e.g., .classname).
- Helpful when you want different formats for multiple elements like <p>.

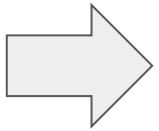
# Example

```html
<!DOCTYPE html>
<html>
<head>
  <title>CSS Classes Example</title>
  <style>
    p.blue {
      color: blue;
    }

    p.red {
      color: red;
    }
  </style>
</head>
<body>

  <p class="blue">Here is the blue class</p>
  <br>
  <p class="red">Here is the red class</p>

</body>
</html>
```



Here is the blue class

Here is the red class

# Same Class for Multiple Tags

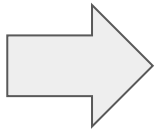
- You can reuse the **same class** (class="classname") across **multiple tags** (e.g., <h1>, <h2>, <p>).
- In the CSS, define the class using a **dot (.)** — e.g., .classname.
- This ensures **consistent styling** across different elements.

# Example

```html
<!DOCTYPE html>
<html>
<head>
  <title>CSS Lectures</title>
  <style>
    .blue {
      color: blue;
    }
  </style>
</head>
<body>

  <h1 class="blue">Here is the blue header1</h1>
  <h2 class="blue">Here is the blue header2</h2>
  <br>
  <p class="blue">Here is the blue paragraph</p>

</body>
</html>
```



**Here is the blue header1**

**Here is the blue header2**

Here is the blue paragraph

# External CSS

- External CSS is written in a **separate .css file**.
- It allows you to **reuse styles** across multiple HTML pages.
- The CSS file contains **only style rules**, no <style> or <head> tags.
- Link it in the HTML <head> using:
  - <link rel="stylesheet" type="text/css" href="filename.css">
- Make sure the .css file is saved in the **same folder** as the HTML file (or provide the correct path).

# Example

HTML File

```html
<!DOCTYPE html>
<html>
<head>
  <title>CSS Lectures</title>
  <link rel="stylesheet" type="text/css" href="Colors.css">
</head>
<body>

  <p class="par1">Here is the normal paragraph</p>
  <p class="par2">Here is the normal paragraph</p>
  <p class="par3">Here is the normal paragraph</p>

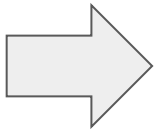
</body>
</html>
```

Colors.css

```css
p.par1 {
  color: blue;
}

p.par2 {
  color: red;
}

p.par3 {
  color: green;
}
```



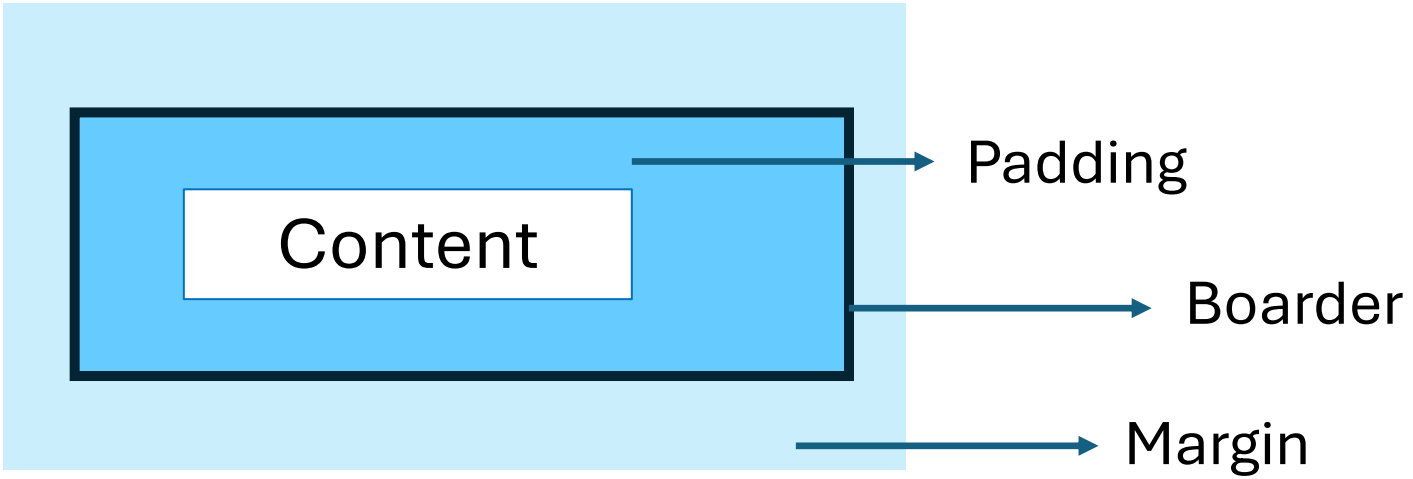
Here is the normal paragraph

Here is the normal paragraph

Here is the normal paragraph

# CSS Box Model

- The box model defines how **HTML elements are rendered** and spaced on the page.
- Every element consists of:
  - **Content** – The actual text or media inside the element.
  - **Padding** – Space **inside the border**, around the content.
  - **Border** – A line that wraps around the padding and content.
  - **Margin** – Space **outside the border**, separating elements from each other or the page edges.
- All values can be applied **individually (top, right, bottom, left)** or **uniformly**.

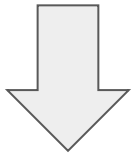


# Example

```html
<!DOCTYPE html>
<html>
<head>
  <title>CSS Box Model</title>
  <style>
    p.par1 {
      color: blue;
      background-color: yellow;      /* Content background */
      padding: 25px;                 /* Space inside border */
      border-color: red;
      border-width: 5px;
      border-style: solid;           /* Red border */
      margin: 60px;                  /* Space outside border */
    }
  </style>
</head>
<body>

  <p class="par1">This paragraph demonstrates the full CSS Box Model.</p>

</body>
</html>
```



This paragraph demonstrates the full CSS Box Model.

# Images with CSS Box

- You can apply **box model styling** to images using a **CSS class**.
- **display: block**: Makes the image behave like a block-level element (so you can center it).
- **margin-left: auto; margin-right: auto;**: Centers the image horizontally.
- **padding**: Adds space between the image and the border.
- **background-color**: Adds a background color behind the image.
- **border**: Adds a border with specified color, width, and style.

# Example

```html
<!DOCTYPE html>
<html>
<head>
  <title>Image with CSS Box</title>
  <style>
    .imgclass {
      display: block;
      margin-left: auto;
      margin-right: auto;
      padding: 10px;
      background-color: silver;
      border-color: blue;
      border-width: 5px;
      border-style: solid;
    }
  </style>
</head>
<body>
  <h2>IIT BHILAI Logo with CSS Box Model</h2>
  <img class="imgclass" src="https://upload.wikimedia.org/wikipedia/en/2/27/IIT_Bhilai_Logo.svg" alt="Logo" width="300">

</body>
</html>
```

IIT BHILAI Logo with CSS Box Model



# CSS Bootstrap

- Bootstrap is an Open-source front-end framework
- Bootstrap includes HTML and CSS based design templates for typography, forms, buttons, tables, navigation, modals, image carousels and many other, as well as optional JavaScript plugins
- Bootstrap also gives you the ability to easily create responsive designs
- https://getbootstrap.com/
- Current version v5.3
- https://getbootstrap.com/docs/5.3/getting-started/introduction/

The world's most popular framework for building responsive, mobile-first sites, with JSDelivr and a template starter page



# CSS Bootstrap: Buttons



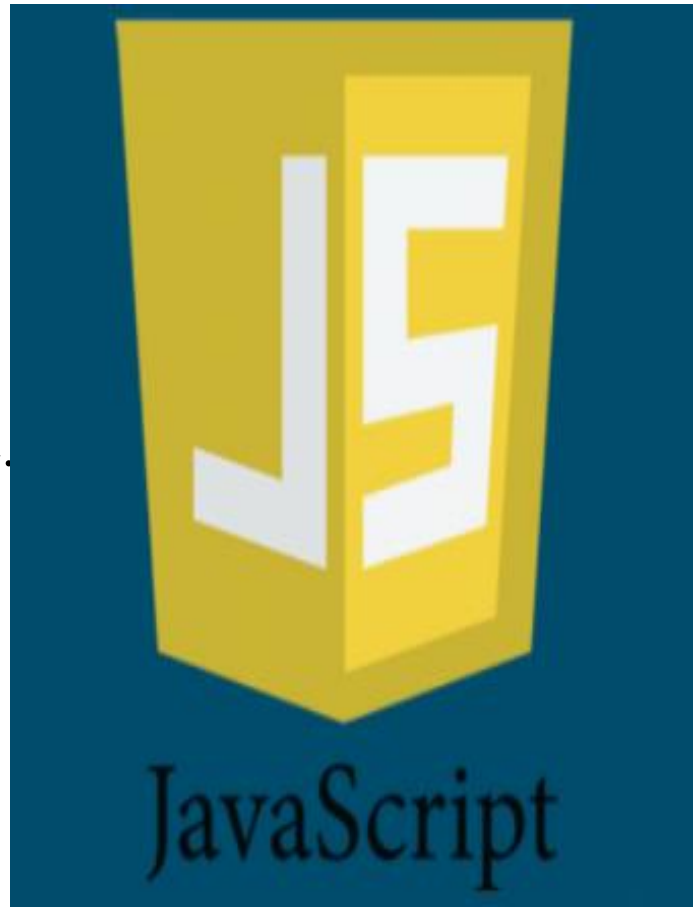

```
<!Doctype html>
<html>
<head>
<link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/5.0.0-alpha1/css/bootstrap.min.css"
integrity= "sha384-r4NyP46KrjDleawBgD5tp8Y7UzmLA05oM1iAEQ17CSuDqnUK2+k9luXQOfXJCJ4I"
        crossorigin="anonymous">
</head>
<body>
      <button class= “btn btn-primary” >  Primary  </button>
      <button class= “btn btn-secondary” >  Secondary  </button>
      <button class= “btn btn-success” >  Success  </button>
      <button class= “btn btn-danger” >  Danger  </button>
      <button class= “btn btn-warning” >  Warning  </button>
</body>
</html>
```

List of Bootstrap classes: https://www.w3schools.com/bootstrap/bootstrap_ref_all_classes.asp

# JAVASCRIPT

# Introduction

- **JavaScript** is the programming language used to **make web pages interactive**.
- It allows you to create **dynamic content**, such as responding to user actions, modifying the DOM, validating forms, etc.
- JavaScript code is written between <script>...</script> tags.
- JavaScript can be included in two ways:
  - **Inline/Internal** – Written directly inside the HTML file.
  - **External** – Written in a separate .js file and linked to the HTML.
- To include an external .js file, use:
  - <script src="code.js"></script>
- The external file (e.g., code.js) must be in the **same folder** or the correct path must be given.



# Example

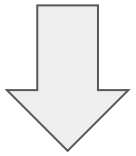
- JavaScript code is placed in an external file (Code.js) for better reusability and cleaner HTML.
- The document.write() method is used to directly write content into the HTML page.
- The .js file is included in the HTML using the <script src="..."> tag inside the <head> or before </body>.
- Make sure the JS file is saved in the **same folder** as the HTML file (or use a correct path).

HTML File

```
<!DOCTYPE html>
<html>
<head>
  <title>Welcome to JavaScript</title>
  <script src="Code.js"></script>
</head>
<body>
```

Code.js

```
document.write("This statement is written inside JS file");
```



This statement is written inside JS file

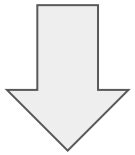
# Variables

- **Variables** store data values like numbers, strings, or Booleans.
- Declaring variables using var, let, or const:
  - var is traditionally used (especially in older code).
- You can perform operations (like addition) using variables.
- document.write() is used to **display output** directly on the webpage.
- The + operator is used to **concatenate** strings and variables.

In HTML File only calling is required

Code.js

```
var x = 4;
var y = 7;
var z = x + y;
document.write("The addition result is " + z);
```



The addition result is 11

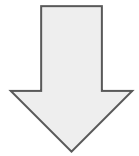
# IF Conditional Statement

- if...else is used to execute different code based on conditions.
- If the condition is **true**, the if block runs.
- If the condition is **false**, the else block runs.
- The condition is usually a **comparison** (e.g., x > y).

Code.js

```
var x = 4;
var y = 7;

if (x > y) {
  document.write("x is greater than y and it is equal to " + x);
} else {
  document.write("y is equal to or greater than x and it is equal to " + y);
}
```



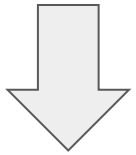
y is equal to or greater than x and it is equal to 7

# For Loop Statement

- A for loop is used for **repeating a block of code** multiple times.
- Syntax includes:
  - Initial value (var i = 1)
  - Condition (i <= 5)
  - Increment (i++)
- Loop runs until the condition becomes false.

Code.js

```
for (var i = 1; i <= 5; i++) {
  document.write("This is the repeated statement...<br>");
}
```



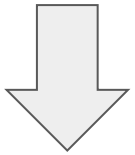
This is the repeated statement...
This is the repeated statement...
This is the repeated statement...
This is the repeated statement...
This is the repeated statement...

# JavaScript Arrays

- An array stores **multiple values** in a single variable.
- Values are stored in **index positions** starting from 0.
- You can access a specific item using array[index]

Code.js

```
var users = ["Soham", "Radha", "Vijay", "Arun"];
document.write(users[0] + "<br>");
document.write(users[2] + "<br>");
document.write(users[1] + "<br>");
document.write(users[4] + "<br>"); // accessing index not defined in array
```



Soham
Vijay
Radha
undefined

# JavaScript Functions

- A **function** is a block of code designed to **perform a specific task**.
- It helps to **reduce code repetition** and **organize logic**.
- A function is:
  - **Defined** once using the function keyword.
  - **Called** when needed using its name followed by parentheses.

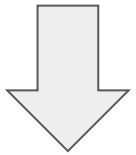
# Example 1: Function That Writes Output to Page

- This function takes two numbers, adds them, and writes the result directly using document.write().

Code.js

```
function add(x, y) {
  var z = x + y;
  document.write("The addition result is " + z + "<br>");
}

add(7, 9); // Calling the function
```



The addition result is 16

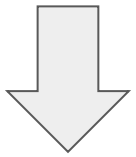
# Example 2: Function That Returns a Value

This version **returns the result** to the code, which can be stored and used later.

Code.js

```
function add(x, y) {
  var z = x + y;
  return z; // Return the result
}

var m = add(7, 8); // Calling the function and storing result
document.write("The addition result is " + m + "<br>");
```



The addition result is 15

# Applying JavaScript with HTML Buttons

- JavaScript can be used to **dynamically change styles** or content when a button is clicked.
- The <button> element uses the **onclick** attribute to call a function.
- The function is defined in an **external JavaScript file** (Code.js) for better structure.
- To target a specific element, use **document.getElementById("id")**.
- You can then access the element's **style** properties (e.g., color, backgroundColor).

# Example

## HTML File

```html
<!DOCTYPE html>
<html>
<head>
  <title>Welcome to JavaScript</title>
  <script src="Code.js"></script>
</head>
<body>

  <p id="text-1">IIT BHILAI</p>
  <button onclick="changeStyle()">Submit</button>

</body>
</html>
```

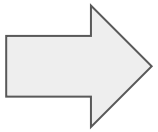
## Code.js

```js
function changeStyle() {
  document.getElementById("text-1").style.color = "blue";
  document.getElementById("text-1").style.backgroundColor = "yellow";
}
```

## Before Click

IIT BHILAI

Submit



## After Click

IIT BHILAI

Submit

# Changing Text Content using innerHTML

- innerHTML is used to change the content of an HTML element dynamically.

HTML File

```html
<!DOCTYPE html>
<html>
<head>
  <title>Welcome to JavaScript</title>
  <script src="Code.js"></script>
</head>
<body>

  <p id="text-1">Old Text</p>
  <button onclick="changeText()">Submit</button>

</body>
</html>
```

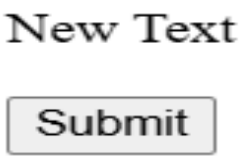
Code.js

```js
function changeText() {
  document.getElementById("text-1").innerHTML = "New Text";
}
```

Before Click




After Click



# Reading Text Content and Copying to Another Element

- This reads text from one paragraph and sets it into another.

HTML File

```
<!DOCTYPE html>
<html>
<head>
  <title>Welcome to JavaScript</title>
  <script src="Code.js"></script>
</head>
<body>

  <p id="text-1">This text will be copied</p><br>
  <p id="text-2">This text will be changed to be like the text above</p>
  <button onclick="changeText()">Submit</button>

</body>
</html>
```

Code.js

```
function changeText() {
  var myText = document.getElementById("text-1").innerHTML;
  document.getElementById("text-2").innerHTML = myText;
}
```

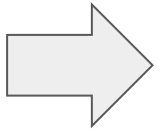
# Example Contd...

Before Click

This text will be copied

This text will be changed to be like the text above

Submit



After Click

This text will be copied

This text will be copied

Submit

# Reading Input Field Value and Copying to Paragraph

- This reads the user's input from a text box and displays it in a paragraph.

HTML File

```html
<!DOCTYPE html>
<html>
<head>
  <title>Welcome to JavaScript</title>
  <script src="Code.js"></script>
</head>
<body>

  <p id="text-1">This text will be copied:</p>
  <input type="text" id="input-1" name="">
  <p id="text-2">This text will be changed to user input</p>
  <button onclick="changeText()">Submit</button>

</body>
</html>
```

Code.js

```js
function changeText() {
  var myText = document.getElementById("input-1").value;
  document.getElementById("text-2").innerHTML = myText;
}
```

# Example Contd...

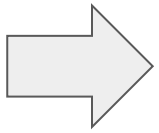
Before Click

This text will be copied:

updated text

This text will be changed to user input

Submit



After Click

This text will be copied:

updated text

updated text

Submit

# Password Validation with If Statement

- Checks if the entered password matches "1234", then shows success or failure.

HTML File

```html
<!DOCTYPE html>
<html>
<head>
  <title>Welcome to JavaScript</title>
  <script src="Code.js"></script>
</head>
<body>

  <p>Please Enter The Password:</p>
  <input type="password" id="input-1" name="">
  <p id="text-1">The login result will be shown here</p>
  <button onclick="checkPassword()">Submit</button>

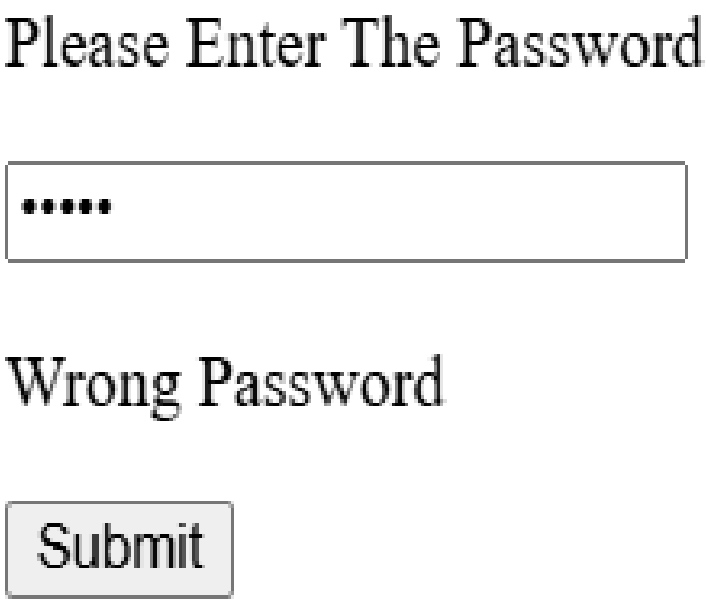
</body>
</html>
```

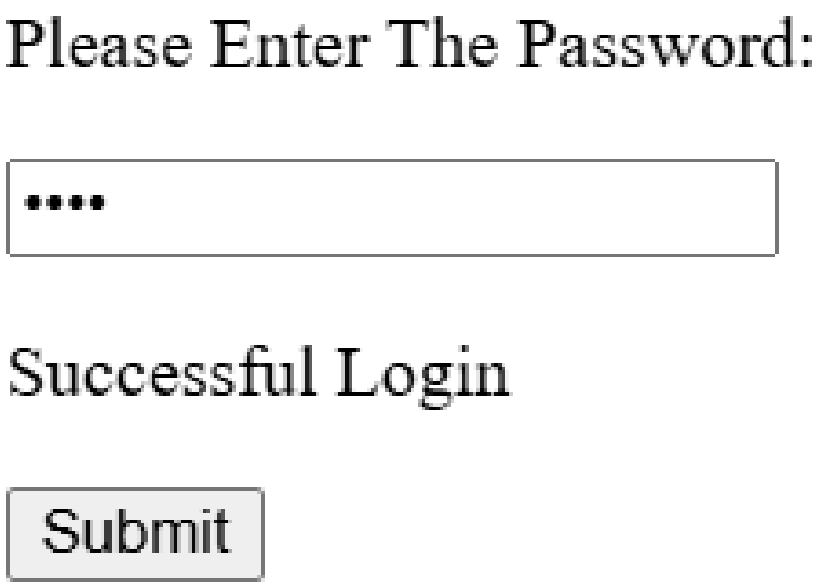
Code.js

```javascript
function checkPassword() {
  var input = document.getElementById("input-1").value;

  if (input == "1234") {
    document.getElementById("text-1").innerHTML = "Successful Login";
  } else {
    document.getElementById("text-1").innerHTML = "Wrong Password";
  }
}
```

# Example Contd...



# Thank you!

Email: anand@iitbhilai.ac.in